DTensorViz

Ryan Le

June 2025

1 Introduction

DTensorViz is a domain specific language designed to help users learn and visualize operations on tensors, particularly operations used in distributed neural network training. DTensorViz provides an intuitive scripting interface for creating, manipulating, and visualizing tensor distributions across multiple devices. Its implementation can be found at https://github.com/RyanLe2003/DTensorViz/tree/main

DTensorViz programs consist of statements that operate on tensors and device groups. The language supports tensor creation, distribution operations (sharding, replication), collective operations (gather, reduce), mathematical operations (matrix multiplication, ReLU), and visualization capabilities.

Conversely, as it currently stands, DTensorViz does no support the following distributing computing scenarios (though may be extended in the future):

- 3D+ tensors: Limited to 2d matrices as intention is more for beginners at the moment
- Complex Topologies: Device groups are limited to rectangular grids

2 Getting Started

A DTensorViz program consists of only statements. Each statement is ended with a semi-colon. Below is a small snippet of DTensorViz code:

```
init_dev(1);
init_dev(2);

let tensor_col_ex = tensor([[1, 2], [3, 4]], dev=1);
visualize(tensor=tensor_col_ex);

let dg_1 = devices([[1, 2]]);
shard(tensor=tensor_col_ex, device_group=dg_1);
visualize(tensor=tensor_col_ex);
```

```
gather(tensor=tensor_col_ex, dst=1);
visualize(tensor=tensor_col_ex);
```

As seen above, DTensorViz is essentially a scripting language with variable declarations and a set of pre-defined functions that can be called on these variables.

3 Language Structure

3.1 Statements

Statements in DTensorViz end with a semicolon. Whitespace (e.g., spaces, indentation) is ignored. There are three types of statements:
- Variable bindings: Assign values to variables using let
- Operations: Built-in functions that manipulate tensors
- Expression statements: Expressions that are evaluated but not assigned

3.2 Expressions

Expressions in DTensorViz evaluate to values. The main expression types are:
- Literals: Tensor literals, device group literals
- Variables: References to previously defined variables
- Operations: Function calls that return values

```
[[1, 2], [3, 4]]
tensor([[1, 2], [3, 4]], dev=1)
devices([[1, 2]])
```

4 Data Types

4.1 Tensors

Tensors are the primary data type in DTensorViz. They are created using the tensor function:

```
tensor([[1, 2], [3, 4]], dev=1)
```

A tensor has:
- Values: A 2D matrix of numbers
- Device: The device where the tensor initially resides
- Distribution state: Whether it's sharded, replicated, or on a single device (this is managed in the backend)

4.2 Device Groups

Device groups define how tensors are distributed across multiple devices:

```
devices([[1, 2]])      // Horizontal layout
devices([[1], [2]])    // Vertical layout
devices([[1, 2], [3, 4]])  // 2x2 grid
```

The shape of the device group determines how tensors are partitioned.

5 Variables

DTensorViz supports storing the result of expression as variables. For example:

```
let variable_name = expression;
```

Variables can store tensors, device groups, or the results of operations. Variable names must start with a letter and can contain letters, numbers, and underscores.

6 Built in Operations

6.1 Device Management

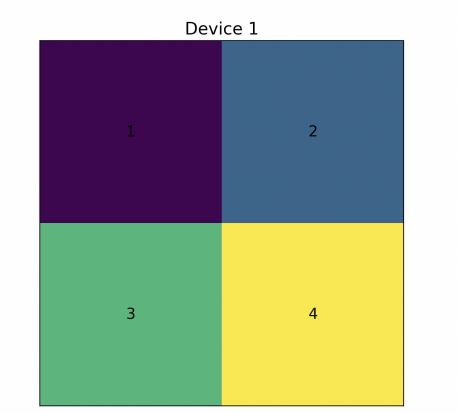
All devices must be initialized before being referenced in device groups.

```
init_dev(1);
init_dev(2);
```

6.2 Visualize

Displays the current state of a tensor. For example:

```
let tensor_ex = tensor([[1, 2], [3, 4]], dev=1);
visualize(tensor=tensor_ex);
```



6.3 Matrix Multiplication

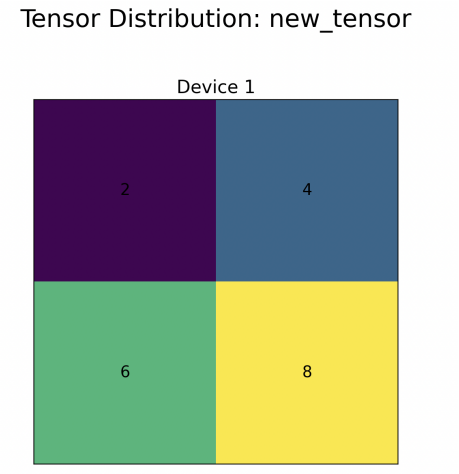
DTensor allows for matrix multiplication between two tensors. It returns a new distributed tensor. For example:

```
init_dev(1);
init_dev(2);

let tensor_one = tensor([[1, 2], [3, 4]], dev=1);
let tensor_two = tensor([[2, 0], [0, 2]], dev=1);

let new_tensor = matmul(tensor_one, tensor_two);
visualize(tensor=new_tensor);
```

Note: As with any form of 2-d matrix multiplication, the inside dimensions of the two tensors must match. Additionally, the two tensors ***must*** *be on the same device.*



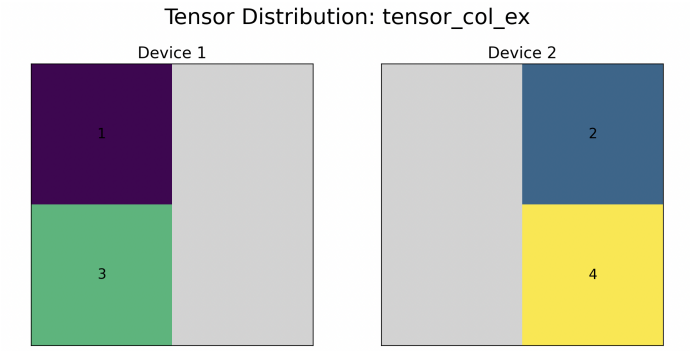
6.4 Shard

DTensor allows a tensor to be sharded across a device group. It is an in place operation on the tensor. For example:

```
init_dev(1);

let tensor_col_ex = tensor([[1, 2], [3, 4]], dev=1);
let dg_1 = devices([[1, 2]]);
shard(tensor=tensor_col_ex, device_group=dg_1);

visualize(tensor=tensor_col_ex);
```

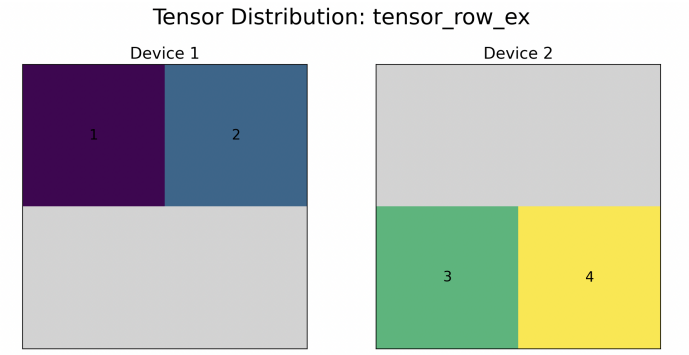


Note: The above example shards a tensor along its columns.

```
init_dev(1);

let tensor_row_ex = tensor([[1, 2], [3, 4]], dev=1);
let dg_2 = devices([[1], [2]]);
shard(tensor=tensor_row_ex, device_group=dg_2);

visualize(tensor=tensor_row_ex);
```



Note: The above example shards a tensor along its rows.

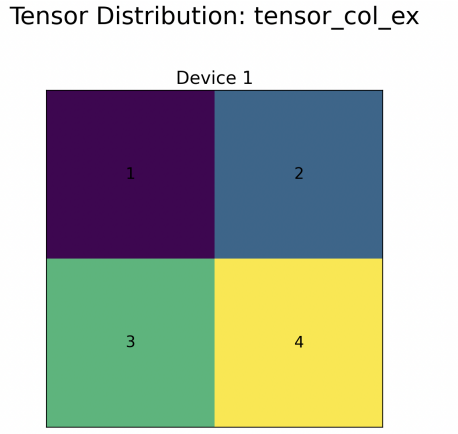
6.5 Gather

DTensor allows a **sharded** tensor to be reassembled onto a single destination device. It is an in place operation on the tensor. For example:

```
init_dev(1);

let tensor_col_ex = tensor([[1, 2], [3, 4]], dev=1);
let dg_1 = devices([[1, 2]]);
shard(tensor=tensor_col_ex, device_group=dg_1);

gather(tensor=tensor_col_ex, dst=1);
visualize(tensor=tensor_col_ex);
```



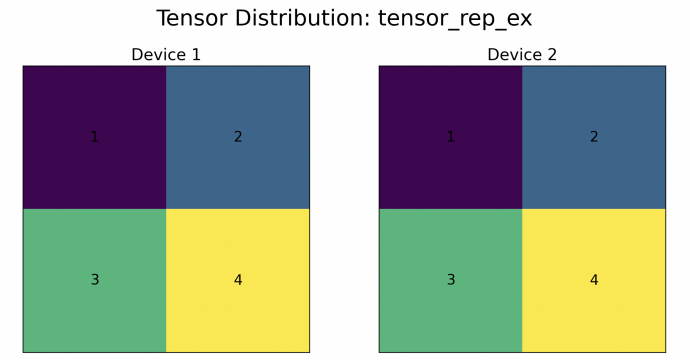
6.6 Replicate

DTensor allows a tensor to be replicated across a device group. It is an in place operation on the tensor. For example:

```
init_dev(1);

let tensor_rep_ex = tensor([[1, 2], [3, 4]], dev=1);
let dg_1 = devices([[1, 2]]);

replicate(tensor=tensor_rep_ex, device_group=dg_1);
visualize(tensor=tensor_rep_ex);
```



6.7 Reduce

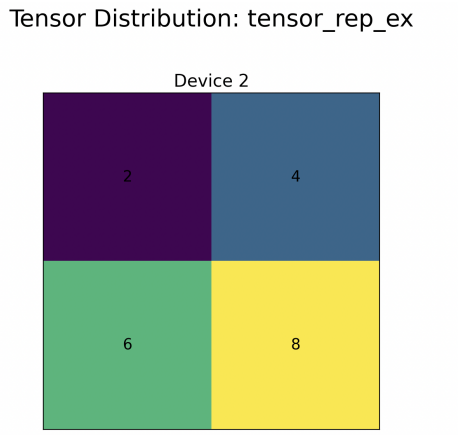
DTensor allows a **replicated** tensor to be aggregated (average) onto a single destination device. It is an in place operation on the tensor. For example:

```
init_dev(1);

let tensor_rep_ex = tensor([[1, 2], [3, 4]], dev=1);
let dg_1 = devices([[1, 2]]);

replicate(tensor=tensor_rep_ex, device_group=dg_1);

reduce(tensor=tensor_rep_ex, dst=2);
visualize(tensor=tensor_rep_ex);
```

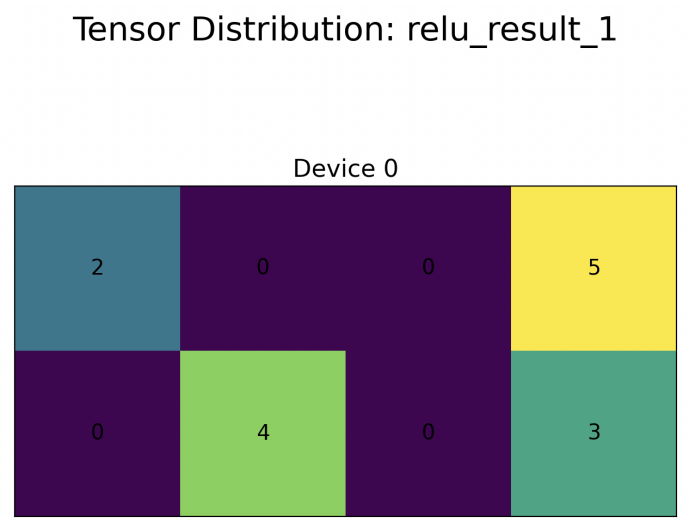


6.8 ReLu

DTensor allows for the ReLU activation function to be used on a single tensor. It returns a new distributed tensor. For example

```
init_dev(0);

let test_tensor_1 = tensor([[2, -3, 0, 5], [-1, 4, -2, 3]], dev=0);
let relu_result_1 = relu(test_tensor_1);
visualize(tensor=relu_result_1);
```



7 Implementation Architecture

DTensorViz is an external dsl (I wanted more practice with externals even though an internal dsl may have allowed for more flexibility here). The DTensorViz implementation follows a traditional interpreter architecture with four main components:

1. Lexical Analysis
2. Parsing
3. Type checking
4. Interpretation

7.1 Lexical Analysis

Uses Parsimonious grammar for tokenization

```
program = (statement / emptyline)*

statement = (binding / operation / simple_expr) semicolon

binding = let name equals expr

operation = shard_op / replicate_op / reduce_op / gather_op / visualize_op / init_device / matmul_op / relu_op

shard_op = "shard" lparen "tensor" equals expr comma "device_group" equals expr rparen
replicate_op = "replicate" lparen "tensor" equals expr comma "device_group" equals expr rparen
reduce_op = "reduce" lparen "tensor" equals expr comma "dst" equals integer rparen
gather_op = "gather" lparen "tensor" equals expr comma "dst" equals integer rparen
visualize_op = "visualize" lparen "tensor" equals expr rparen
matmul_op = "matmul" lparen expr comma expr rparen
relu_op = "relu" lparen expr rparen

init_device = "init_dev" lparen integer rparen

expr = operation / simple_expr
simple_expr = tensor_literal / device_group_literal / variable

tensor_literal = "tensor" lparen tensor_data comma "dev" equals integer rparen
tensor_data = lbracket (tensor_row comma)* tensor_row rbracket
tensor_row = lbracket (number comma)* number rbracket

device_group_literal = "devices" lparen device_group_data rparen
device_group_data = lbracket (device_row comma)* device_row rbracket
device_row = lbracket (integer comma)* integer rbracket

variable = name

number = float / integer
float = ~r"-?[0-9]+\.[0-9]+" ws
integer = ~r"-?[0-9]+" ws

let = "let" ws
name = ~r"[a-zA-Z]\w*" ws
equals = "=" ws
comma = "," ws
semicolon = ";" ws
lparen = "(" ws
rparen = ")" ws
lbracket = "[" ws
rbracket = "]" ws

emptyline = ws+
ws = ~r"\s*"
```

Figure 1: DTensorViz syntax

7.2 Parsing

Builds an Abstract Syntax Tree (AST) using a visitor pattern

```
class TensorDSLVisitor(NodeVisitor):
    def visit_shard_op(self, node, visited_children):
        tensor = visited_children[4]
        device_group = visited_children[8]
        return Shard(tensor, device_group)

@dataclass
```

```
class Shard(Statement):
    tensor: Expr
    device_group: Expr
```

7.3 Type Checking

Verifies the data types of variables and expressions. Follows the visitor pattern of traversing AST nodes and throws a TypeError immediately on mismatches.

7.4 Interpretation

The runtime maintains global device state and tracks tensor distribution properties. It utilizes the DistributedTensor class to perform the parallel operations outlined above.

7.4.1 DistributedTensor Class

The DistributedTensor class contains:
- Full tensor data: Complete tensor values for visualization
- Device mapping: Dictionary mapping device IDs to tensor chunks
- Distribution state: Tracking whether tensors are sharded, replicated, or on single devices
- Device group structure: Current device layout for operations

It also contains class functions that perform the parallel operations manually.

```
class DistributedTensor:
    def __init__(self, tensor=None, device=None):
        self.full_tensor = tensor
        self.device_map = {device: self.full_tensor}
        self.cur_dev_group = [[device]]
        self.is_shard = False
        self.is_replicated = False
```

Note: is_shard and is_replicated can also be changed by algebraic operations (not just parallel). The table below outlines how a matmul operation will change the state of the tensor

Tensor One	**Tensor Two**	**Result**
Replicated	Replicated	Replicated
Shard	Shard	Replicated
Shard	Replicated/None	Shard
Replicated/None	Shard	Shard
None	None	None

8 Related Work

DTensorViz draws inspiration from PyTorch's distributed tensor framework, as it is arguably the most accessible machine learning library. However, it address's some of PyTorch's shortcomings. In particular, for someone who is just getting started with distributed training, the most important thing is arguably understanding how these operations actually distribute and change tensors. The lowest level of torch.distributed expects too much out of the programmer, and at the highest level (DTensor), the language abstracts away too much to really learn. Additionally, while the programmer actually needs multiple devices to see the true benefits of distributed training, when learning the operations, this isn't actually needed. There isn't a super clean way for a beginner to test distributed operations on a single device without extra setup.

9 Evaluation

DTensorViz successfully addresses its primary goal of making distributed tensor concepts accessible:
- Immediate Feedback: Users see the effects of operations instantly
- Conceptual Clarity: Visual representations clarify abstract distribution concepts
- Experimentation: Low barrier to trying different distribution strategies
- Error Understanding: Runtime errors provide clear explanations of invalid operations

All of which was confirmed through feedback from classmates and individuals who work in the domain.